Introduction to the Strategy of Positive/Negative: Text Polarity Classification Kaggle Competition

PO-LIAN LIAO
114423053
williamliao2013@gmail.com

ABSTRACT

This project is for a "Positive/Negative Sentiment Classification" competition. Only 2,000 labeled records and 11,000 unlabeled test records were available. The goal of this research was to build a model that can reliably predict the sentiment of short English texts.

In the data preprocessing step, this study fixed broken abbreviations (like "didn t") and numbers in the text. This made sure important emotion words, like negative words, were not deleted by mistake. For the models, the experiment started with a basic TF-IDF method. Then, pre-trained language models like DistilBERT, RoBERTa, DeBERTa, and ELECTRA were introduced. 5-Fold cross-validation was also used to make the training results stable.

To make good use of the unlabeled test data, this research used a two-step self-training method. In step one, a starting model was trained using the 2,000 labeled records. In step two, the model's most confident guesses on the test set were selected as "pseudo labels." These were mixed with the original data to train the model again. The weight of these pseudo labels was carefully controlled so that errors would not build up. Finally, instead of using a fixed cut-off point, a threshold search was applied to make the predictions more accurate.

For the final submission, the results of three models were averaged: RoBERTa-large, DeBERTa-v3-large, and ELECTRA-large. Because these three models were trained differently, they made different kinds of mistakes. This helped lower the total prediction error. In the end, the cross-validation (OOF) F1 score reached 0.8864. The score on the Kaggle Public Leaderboard was 0.79807, and the final Private Leaderboard score was 0.81099. The experiment also showed that doing more rounds of self-training made internal scores higher, but leaderboard scores actually went down. Therefore, this study concluded that "using different kinds of models is better than repeating self-training on a single model."

1. INTRODUCTION

1.1 Task Definition

The task of this study is to classify the sentiment of short English texts into two categories: positive (marked as 1) and negative (marked as 0). This dataset presents two main challenges. First, the texts are short clips from many different sources, separated from their original context. The competition rules also state that there is no absolute boundary between positive and negative emotions, which makes the classification naturally fuzzy. Second, the training set contains only 2,000 human-labeled records, while the test set has 11,000 records (a ratio of about 1:5.5). This means the amount of data available for training is very small.

In addition, the competition uses a two-stage scoring system. The Public Leaderboard is based on only 33% of the test data, while the final ranking is decided by the complete Private test set. Therefore, building a model that can make stable predictions on new, unseen data is much more important than just getting a high score on the Public Leaderboard.

1.2 Research Contributions

To deal with the challenges mentioned above, this study designed a complete experiment plan, covering data cleaning, model selection, training strategies, and the final integration. Since the labeled data was limited and there was much more test data than training data, the core problem of this research was how to make the model perform well not only on the training set but also make stable predictions on new, unseen data.

The four main contributions of this study are as follows:

- Comparing the performance of three different stages: the Bag of Words model, a single Transformer, and a multi-model mix (Ensemble). This showed the actual improvement brought by each stage.
- Introducing a two-step self-training plan. A starting model was trained with the labeled data first. Then, the model's most confident guesses on the test set were selected as extra training labels and mixed with the original data for retraining. Also, a threshold search was used instead of a fixed cut-off point. This steadily improved the predicting ability using only 2,000 labeled records (see Section 3.3 for details).
- Creating the "OOF-Public Gap" marker to measure the difference between cross-validation scores and actual leaderboard scores. Through experiments, it showed the risk that too much self-training can make this difference larger (see Section 4.4 for details).
- Choosing three language models with different training backgrounds to form the Ensemble. This proved that using different model types can lower the chance of multiple models making the same mistake at the same time, which improved the overall prediction accuracy (see Section 3.4 for details).

2. DATA PREPROCESS

2.1 Introduction

This chapter explains the complete data cleaning process, from the raw competition data to the inputs ready for the model. This dataset includes 2,000 training records and 11,000 test records. Each record has three columns: row_id, TEXT, and LABEL (only in the training set). The label is a two-way classification: 1 means positive and 0 means negative. The ratio of the two classes is about 1:1.

Three main challenges shaped the design of this process. First, the data was only half-processed. The apostrophes in abbreviations were replaced by spaces (like "didn t" and "it s"), and numbers were replaced by placeholders (like "num_num" and "num_extend"). The text could not be treated as raw strings or assumed to be separated into words yet. Second, the training data was very small. With only 2,000 original labels, the model could easily overfit. Therefore, semi-supervised methods were needed to fully use the 11,000 test records. Third, the leaderboard was split into Public and Private. The Public Leaderboard only used 33% of the test data, but the final ranking was decided by the Private one. Thus, the ability

to make good predictions on new, unseen data was much more important than just getting a high score on the Public Leaderboard.

Based on these challenges, this process followed three rules. The first rule was to set cleaning steps that keep important emotion words, especially negative words. The second rule was to use different levels of cleaning for different types of models, such as strong cleaning for the Bag of Words model and weak cleaning for Transformer models. The third rule was to strictly keep all data-related steps inside the K-Fold process to prevent data leaking. The following sections will explain these in order: Section 2.2 Exploratory Data Analysis, Section 2.3 Types of Text Noise, and Section 2.4 Preprocessing Pipeline.

2.2 Exploratory Data Analysis

2.2.1 Class Distribution

As shown in Figure 1, the training set contains exactly 1,000 negative (label=0) and 1,000 positive (label=1) records. The categories are perfectly balanced. Because of this, there is no need to use special methods like "class weight" to fix unbalanced data during the training stage. On the other hand, if the model predicts a positive rate that is very far from 0.500 during the prediction phase, it is a warning sign that the probabilities are not balanced. This observation serves as the main reason for adjusting the cut-off point (threshold) later in the cross-validation (OOF) process.

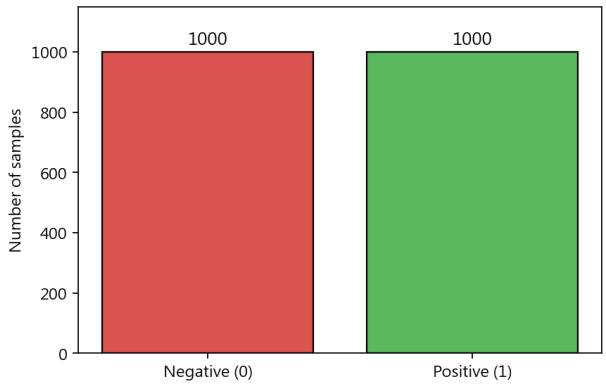

Figure 1. Training set class distribution

2.2.2 Text Length Distribution

As shown in Figure 2, the text length of the training set and the test set are very similar (with an average of about 19 to 20 words). Most of the records are shorter than 60 words, and the longest one is no more than 70 words. This finding supports the decision to set the max_length of the Transformer tokenizer to 128. This setting covers over 99% of the records and leaves enough space for punctuation marks and sub-word splitting.

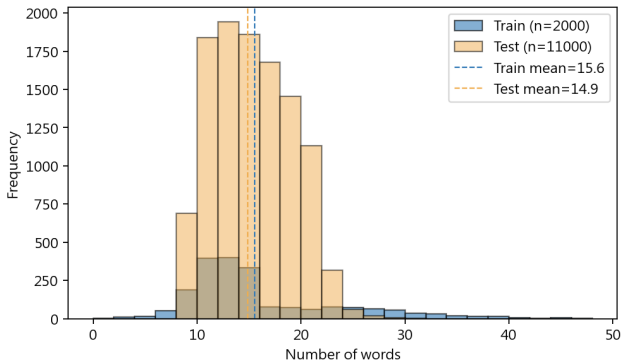

Figure 2. Distribution of text length in training and test sets

2.2.3 Lexical Polarity Analysis

This study further used word frequency counting to see how much information the Bag of Words model can actually catch. The ratio of appearing in the positive category was calculated for the top 30 most frequent words in the training set (after removing stop words and short words). The results are shown in Figure 3: words like "great", "well", "use", and "into" are clearly positive (>0.55, green). Words like "too", "more", "even", "movie", and "been" are clearly negative (<0.45, red). The middle part contains nearly neutral words (gray).

The key thing to notice here is that "good" falls into the neutral area (ratio ≈ 0.46), and "like" also leans toward negative (ratio ≈ 0.43). This shows that in this dataset, "good" and "like" appear a lot in negative sentences (such as "not good enough" or "would like to but..."). Their true emotion highly depends on the context. This is exactly the kind of deep meaning that the basic TF-IDF model has trouble catching. It also explains the experiment result later in the version 3 stage, where the cross-validation (OOF) F1 score jumped by +0.15 after introducing better semantic language models.

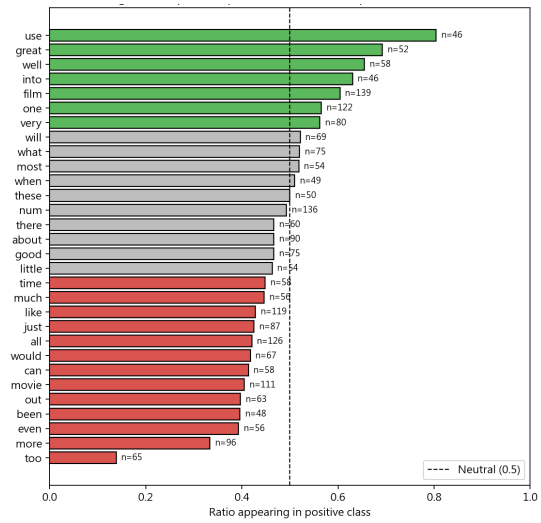

Figure 3. The proportion of high-frequency words in the training set appearing in the positive category

2.3 Text Noise Patterns

After checking the actual training set, the study found the following four types of noise:

- **Broken abbreviations:** Such as "didn t", "it s", "i m", and "don t". The apostrophes were replaced by spaces. This caused the tokenizer to wrongly split negative words and verbs.
- **Special placeholders:** "num_num" and "num_extend" replaced the original numbers. If these were put directly into the tokenizer, they would be cut into many meaningless sub-words.
- **Irregular spaces around punctuation:** There were extra spaces between words and commas or periods. Possessive parts like "'s" were separated from their original words.
- **Big differences in text length:** The lengths ranged from 5 words to over 60 words. A standard cut-off step was needed during the tokenizer stage.

2.4 Preprocessing Pipeline

This study went through six main versions (Version 1 to Version 6). Versions 1 and 2 used the Bag of Words model. Versions 3 to 5 slowly introduced Transformer models. Version 6 focused on ways

to make the model work better on new, unseen data, and it was further updated into sub-versions from Version 6.1 to Version 6.6. In the end, the final submission used a mix of three models (Ensemble) from Version 6.6. The detailed performance comparison of all versions can be found in Chapter 5.

Because the Bag of Words model counts how often words appear, and the Transformer tries to understand the deep meaning of the text, they need very different data cleaning steps. Therefore, this research used different methods for different types of models.

2.4.1 TF-IDF Pipeline (version 1, version2)

For the Bag of Words model, this study designed a four-step cleaning process. The first step was to fix abbreviations by using regular expressions to change "didn t" to "didnnot" and "it s" to "its". The second step was to protect special placeholders by changing "num_num" to "TOKEN_NUM" and "num_extend" to "TOKEN_EXT" so they would not be broken into pieces. The third step was to make all letters lowercase and remove all punctuation except for letters and numbers. The fourth step was to create a custom list of stop words to keep important emotion words like not, no, but, however, never, don, and didn.

There is a specific reason why "didn t" was changed to "didnnot". If it was changed to two words, "did not", the word "not" would still be kept as an emotion word. However, separating the verb and the negative word would make it harder for the Bag of Words model to recognize them. By combining them into one single word, "didnnot", the "past-tense negative" becomes an independent feature. This also prevents the letter "t" from being seen as a useless word and removed by the stop word system.

2.4.2 Transformer Pipeline (version 3 ~ 6)

For the Transformer models, this study only did basic text stripping and Unicode normalization. It used the model's built-in AutoTokenizer (from HuggingFace Transformers[1]) to change the text into sub-word token IDs. The maximum length was set to 128, and both truncation and padding were turned on. Because pre-trained models can already directly handle original text that contains apostrophes and abbreviations, cleaning the data too much would actually destroy the deeper meaning of the text.

3. MODEL SELECTION

This chapter explains the basic algorithms used in this study in the following order: Section 3.1 covers the basic Bag of Words model (Versions 1 and 2). Section 3.2 introduces the Transformer models (Versions 3 to 5). Section 3.3 explains the framework designed to handle new, unseen data better (Version 6 series). Section 3.4 describes the final mix of three models, or Ensemble (Version 6.6). All methods include original references so they can be easily tracked. Please note that testing methods, such as evaluation markers, Cross-Validation, and Soft Voting, are explained together in Sections 4.4 and 4.5. This chapter focuses only on how the models were chosen and built.

3.1 Baseline Models (Version 1, Version 2)

3.1.1 TF-IDF Feature Representation

Term Frequency-Inverse Document Frequency (TF-IDF) [2]measures how important a word is to a document by multiplying the term frequency (TF) by the inverse document frequency (IDF). The main idea is simple: if a word appears very often in one specific document, but rarely in all the other documents, it gets a higher score. On the other hand, words that commonly show up in many documents (like stop words) are given a lower score.

3.1.2 Version 1：Word 1–2 gram TF-IDF + Logistic Regression

For the model features, this study used Word unigram and bigram TF-IDF, with a limit of 5,000 maximum features. The classifier chosen for this task was an L2 regularized Logistic Regression. To check the results, a 5-Fold cross-validation method was applied. The main purpose of this setup was to create a basic standard that could be easily repeated, which served as the lowest passing mark for the Public Leaderboard. It also helped to measure the maximum amount of useful information that the Bag of Words model could actually catch.

3.1.3 Version 2: Expanded Word + Character N-gram

For the model features in this version, this study used Word 1 to 2 grams and Character 1 to 4 grams TF-IDF, creating a total of 15,000 dimensions. Using character n-grams provides better tolerance for spelling changes, such as the created word "didnnot". However, the observation showed only a tiny +0.0075 increase in the cross-validation (OOF) F1 score compared to Version 1. This result proved that the Bag of Words model had almost reached its maximum limit on this dataset. To make further breakthroughs, the research had to rely on distributed semantic representation to help the model understand the deeper meanings of the text.

3.2 Transformer Models (Version3 ~ 5)

The Transformer model uses a method called "self-attention" to replace older, step-by-step calculations.[3] This allows the model to process a whole sentence at the same time and understand connections between words that are far apart. BERT started a new way of learning called "pre-training plus fine-tuning.[4]" First, it learns from a huge amount of text by guessing hidden words (Masked Language Modeling). Then, it makes small adjustments using a small amount of labeled data for specific tasks. This method works extremely well for tasks with very little data, just like the 2,000 training records used in this study.

3.2.1 Version 3: DistilBERT Fine-tuning

DistilBERT uses a method to make the original BERT model 40% smaller, but it still keeps about 97% of its good performance[5].

- **Motivation:** To use a smaller Transformer to quickly see if understanding deep meaning (semantic representation) works well.
- **Result:** The OOF F1 score jumped from 0.6697 to 0.8196 (+0.1499). This was the biggest single jump among all versions.
- **Problem:** Version 3 had a clear overfitting issue. The training error went down to 0.75, but the testing error actually went up.
- **Fix (Version 3.1)**: By using careful data cleaning, Dropout, Weight Decay, and early stopping, the error went down to 0.45, and the OOF F1 score went up to 0.8239.

3.2.2 Version 4: RoBERTa-base + Stratified 5-Fold + Soft Voting

RoBERTa[6] improves how BERT is trained (such as using larger data batches, changing masks dynamically, removing the next sentence prediction task, and training for a longer time). It is steadily better than BERT on many tests.

- **Design:** This version trained RoBERTa-base using a 5-Fold method. During prediction, it averaged the probability scores from all 5 models (this is called soft voting).
- **Result:** The OOF F1 score was 0.8348, and the Public Score was 0.74683. The score differences between the 5 models

dropped, making the model much more stable when handling new data.

3.2.3 Version 5: RoBERTa-large + Pseudo-labeling + FP16

Pseudo-Labeling is a semi-supervised learning method. It takes the model's most confident guesses on unlabeled data as "pseudo labels" and trains them together with the real labels. This works very well when the training data is small but there is a lot of unlabeled test data.

FP16 Mixed Precision[7] changes some calculations to use less memory. It cuts memory use by about 50% and speeds things up by 1.5 to 2 times. This makes it possible to run large models on a single graphics card (GPU).

- **Design:** The Version 4 model's most confident guesses on the 11,000 test records were used as pseudo labels. These were mixed with the original real labels to train the RoBERTa-large model.
- **Result:** The OOF F1 score reached 0.8715 (+0.037 compared to Version 4), and the Public Score was 0.74710. However, the gap between the OOF and Public scores grew to 0.124. This showed that the pseudo labels were starting to bring in errors, meaning more control methods were needed (which led to the Version 6 series).

3.3 Generalization-Oriented Framework (Version 6)

Although Version 5 used pseudo-labeling and increased the cross-validation (OOF) F1 score, the gap between the OOF and Public scores also got bigger. This showed that the pseudo labels made the model too confident during internal testing, but it could not really perform well on new, unseen test data. Therefore, the goal of the Version 6 series changed from "getting high OOF scores" to "making the OOF evaluation match the Public score." To do this, this study introduced five new methods:

- **Two-Stage Self-Training:** In the first stage, the model was trained using only the 2,000 human-labeled records. This allowed the model to build a solid foundation on clean data. In the second stage, the carefully chosen pseudo labels were added to train the model again. This made sure the pseudo labels only worked after the model already had a good start, preventing them from messing up the training direction from the very beginning.
- **Confidence and Margin Filtering:** From the 11,000 test records, only the ones the model was "very sure" about were kept as pseudo labels. The exact rules were: the chance of the top category had to be over 0.92, and the difference between the positive and negative chances had to be over 0.35 to ensure the decision was clear. At the same time, an equal number of positive and negative labels were chosen to keep the categories balanced.
- **Soft Pseudo Labels with Loss Re-weighting:** The pseudo labels were put into the training as chance distributions (like 0.96 and 0.04) instead of strict 0s and 1s to keep the model's uncertain information. Also, the weight of the error (loss) for these pseudo records was set to 0.3. This made their influence lower than the real labeled data, stopping a few noisy pseudo labels from controlling the model too much.
- **OOF Threshold Calibration:** For the Transformer models, 0.5 is not always the best cutting point to decide positive or negative. This is especially true after pseudo-labeling, where the numbers might shift. After the second stage of training, this research searched for the best cut-off point by testing numbers in steps of 0.01. This replaced the fixed 0.5 cut-off, allowing the decision line to automatically match the actual numbers.
- **Label Smoothing:** During training, the labels were slightly smoothed from strict 0s and 1s. This means the correct category became 0.95 and the wrong category became 0.05. This prevented the model from being too confident about the training data, which could hurt its ability to handle new data.

3.4 Final Ensemble Architecture

d Even if a single model gets a high cross-validation (OOF) score, it might still make the same kinds of mistakes on certain types of data. In the end, this study used a mix of multiple models, called an Ensemble. The main goal was to combine the guesses from different models to cover each other's blind spots. This helps improve the model's overall ability to handle new, unseen test data.

How well an Ensemble works depends on the models having a low chance of making the exact same errors[8]. If all the models make mistakes on the same group of data, simply taking the average will not improve the final result. Because of this, this research chose three models that were trained differently in their early stages. This strategy was used to make sure that the types of data each model easily gets wrong do not overlap too much, please look at Table 1.

Table 1. Three models of the Ensemble method.

Model	Pre-training Mechanism	Reason for Choosing
RoBERTa-large[6]	Masked Language Modeling (MLM)	It is the industry standard baseline, and its training process is very stable.
DeBERTa-v3-large[9]	MLM + Disentangled Attention	It separates the calculations for word content and word position. This makes it much better at noticing negative words and sentence order.
ELECTRA-large-discriminator[10]	Replaced Token Detection (RTD)	It checks whether each word has been swapped out. Because every single word is part of the training, it learns much faster than the regular MLM method.

During the prediction stage, the chances of the positive category from the three models for the same record are simply averaged together. Then, the final classification is decided by using the cut-off point adjusted from the cross-validation (OOF) process (tau* = 0.610). The experiment results show that giving the models equal weights (OOF F1 = 0.8864) is better than using a weighted method like 0.4/0.4/0.2 (which scored 0.8860). When the amount of data is limited, adjusting weights based on OOF scores can easily cause the model to memorize the validation set too much. Keeping the weights equal makes the model much more stable when handling new data.

Figure 4 shows the whole development process from Version 1 to Version 6.6. The OOF F1 score jumped from 0.66 with the basic Bag of Words model to 0.82 with a single Transformer model. Finally, it reached 0.8864 by using the three-model Ensemble. This clearly shows the actual improvement brought by each stage.

Figure 5 shows the Ensemble design of Version 6.6. Three models with different early training backgrounds each output their chances for the positive category. After taking the equal average, the OOF adjusted cut-off point (tau = 0.610) is applied to make the final prediction.

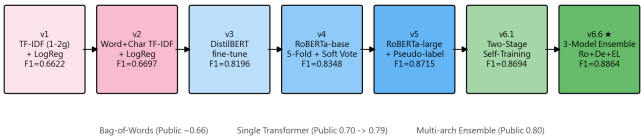

Figure 4. Method evolution pipeline from Version 1 to 6.6.

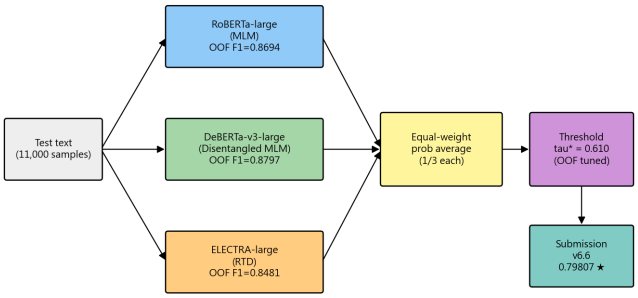

Figure 5. Version 6.6 Three-model Ensemble architecture.

4. EXPERIMENTAL PROCESS

The goal of this study is to build a two-way classification model that can work stably on small amounts of data and noisy text. To do this, this research designed a two-step experiment plan called "Two-Stage Self-Training + Three-Model Ensemble." It also strictly used K-Fold cross-validation and OOF threshold calibration to prevent data leaking and overfitting. This chapter explains the following parts in order: 4.1 Training and Validation Data Segmentation, 4.2 Training Procedure, 4.3 Hyperparameter Settings, 4.4 Evaluation Metrics, 4.5 Cross-Validation Mechanism, 4.6 Ablation Study, and 4.7 Final Ensemble Configuration.

4.1 Training & Validation Data Segmentation

Because the training data in this study only has 2,000 records, using a single hold-out split (like 80/20) could easily make the evaluation unstable due to biased sampling. Therefore, this research used k-fold cross-validation throughout the whole process to check the model's ability to handle new data. It used the following three settings to keep the experiment consistent:

- **Stratified Sampling:** Using StratifiedKFold from sklearn to do stratified sampling. This made sure the ratio of positive to negative records was the same (50%/50%) in every training and validation fold. This stopped evaluation errors caused by unbalanced data categories.
- **5-Fold Validation:** Considering the calculation time for fine-tuning the Transformer models (about 4 to 5 minutes per fold for a large model), 5-Fold was used instead of 10-Fold. With this setting, each fold has 1,600 training records and 400 validation records. This still gives stable OOF estimates even with a small dataset.
- **Reproducibility:** The random_state was fixed at 42 to ensure the data splits were exactly the same for all versions. This made it easier to compare the results of ablation studies across different versions.

4.2 Training Procedure

The core training plan for this study is Two-Stage Self-Training. The first stage only trained the model with the 2,000 human-labeled records. The second stage trained the model again using the human-labeled records mixed with the carefully chosen pseudo labels. This design allowed the Stage 1 model to first build a solid foundation. Then, it used high-quality pseudo labels to make the decision boundaries stronger. This prevented the pseudo labels from taking over and controlling the training process, which could easily happen if everything was mixed together all at once.

To prevent data leaking, all training steps inside each fold, including tokenizer fitting, optimizer states, and schedulers, were run completely independently. The final predicted chance for the test set was calculated by taking the simple average of the five fold models. To make the model better at handling new data, Stage 2 introduced a few extra settings. These included label smoothing set at 0.05, early stopping based on the validation set's macro-F1 score, and the AdamW optimizer with a linear warmup (with a warmup ratio of 0.06).

For the large models, which have 24 layers and 355 million parameters, this research used FP16 mixed precision training. This method cut the memory use by 50% and made the training about 1.5 to 2 times faster. However, there was one exception. The DeBERTa-v3-large model had calculation errors (showing a "NaN loss") when using FP16 because of its special attention calculation method. Therefore, it was trained using regular FP32 instead.

4.3 Hyperparameter Settings

Hyperparameter settings have a huge impact on the final performance when fine-tuning Transformer models. Considering the differences in the early training methods of the three models used in this study (MLM, Disentangled MLM, and RTD), this research applied different settings to each of them. A small-scale manual search was used to find the best values, instead of using automatic search tools like Optuna. This choice was made to prevent the model from memorizing the validation set too much (overfitting), especially since the amount of data was very small. The final parameter settings are as Table 2-5.

The common learning rate for ELECTRA is usually half of what other Transformer models use. When it was first trained with a learning rate of 1e-5, Fold 3 crashed during Stage 1. Its F1 score dropped to 0.030, meaning the model broke down and only guessed one single category. As a result, the cross-validation (OOF) F1 score for Stage 2 was only 0.7801. The main reason for this problem was that ELECTRA's early training part, called the discriminator head, calculates updates differently than the MLM heads used in RoBERTa and DeBERTa. Because of this difference in scale, the standard learning rate was too strong for some folds at the very beginning. After changing the learning rate to 5e-6, all folds became stable, and the OOF F1 score successfully went up to 0.8481.

Table 2. Two-Stage Self-Training Hyperparameters.

Parameter	Setting Value	Search Range
confidence_threshold	0.92	0.90–0.96
margin_threshold	0.35	0.25–0.45
pseudo_loss_weight	0.30	0.20–0.80
max_pseudo_per_class	3,000	—
threshold_grid_step	0.01	—
random_state	42	—

Table 3. Hyperparameter settings for RoBERTa-large.

Parameter	Setting Value	Description
learning_rate	1e-5	The standard learning rate for fine-tuning a Transformer model.
batch_size × grad_accum	8 × 2 = 16	Limited by the 24 GB GPU memory, grad_accum is used to simulate a batch size of 16.
epochs (Stage 1 / Stage 2)	2 / 3	Stage 2 has one extra epoch to fully learn from the pseudo labels.
weight_decay	0.01	The default setting for AdamW.
warmup_ratio	0.06	Does a linear warmup for the first 6% of the steps.
label_smoothing	0.05	Lowers overconfidence and helps the model work better on new data.
fp16	True	Turns on mixed precision training.
max_length	128	Covers 99% of the record lengths.

Table 4. Hyperparameter settings for RoBERTa-large.

Parameter	Setting Value	Description
learning_rate	1e-5	The standard learning rate for fine-tuning a Transformer model.
batch_size × grad_accum	8 × 2 = 16	Limited by the 24 GB GPU memory, grad_accum is used to simulate a batch size of 16.
epochs (Stage 1 / Stage 2)	2 / 3	Stage 2 has one extra epoch to fully learn from the pseudo labels.
weight_decay	0.01	The default setting for AdamW.
warmup_ratio	0.06	Does a linear warmup for the first 6% of the steps.
label_smoothing	0.05	Lowers overconfidence and helps the model work better on new data.
fp16	False	The special attention calculation in DeBERTa-v3 is not stable when using FP16, so it is changed to FP32.
max_length	128	Covers 99% of the record lengths.

Table 5. Hyperparameter settings for RoBERTa-large.

Parameter	Setting Value	Description
learning_rate	1e-5	The gradient scale of the discriminator is larger, so it needs to be cut in half.
batch_size × grad_accum	8 × 2 = 16	Limited by the 24 GB GPU memory, grad_accum is used to simulate a batch size of 16.
epochs (Stage 1 / Stage 2)	2 / 3	Stage 2 has one extra epoch to fully learn from the pseudo labels.
weight_decay	0.01	The default setting for AdamW.
warmup_ratio	0.06	Does a linear warmup for the first 6% of the steps.
label_smoothing	0.05	Lowers overconfidence and helps the model work better on new data.
fp16	True	ELECTRA supports FP16.
max_length	128	Covers 99% of the record lengths.

4.4 Evaluation Metrics

To fully evaluate the model's performance, this study used four different markers. Their definitions are explained below.

4.4.1 Macro-F1 Score

he main marker for this task is the macro-averaged F1 score. In a two-way classification, the F1 score (which is a balance between precision and recall) is calculated for both the positive and negative categories separately. Then, the simple average of these two scores becomes the final grade. This marker is sensitive to unbalanced categories and stops the model from favoring the larger category. It is also the main grading standard for the competition's leaderboard.

4.4.2 Accuracy

This is the percentage of all guesses that match the real answers. Since the categories in this dataset are balanced, accuracy serves as a helpful secondary marker. It is used alongside the F1 score to make sure the guesses do not strongly favor one specific category.

4.4.3 Out-of-Fold (OOF) F1 Score

The OOF guesses are a helpful extra result of the K-Fold method. The predicted chances from each validation fold (which the model has not seen during training) are collected and put together to form a full list matching the size of the entire training set. The OOF F1 is the macro-F1 score calculated from these OOF guesses. Because every single record is guessed by a model that was not trained on it, this score is much closer to how the model will actually perform on new data. It is the main offline evaluation marker for this study.

4.4.4 OOF–Public Gap

This gap is the difference between the OOF F1 score and the Public Leaderboard score. It shows how much the cross-validation estimate differs from the real ranking. A smaller gap means the OOF evaluation is closer to the true performance, showing that the model's ability to guess unseen data is stable. If the gap grows

unusually large, it is a warning sign that the model is memorizing the training data too much or that the OOF evaluation is off. This study used this gap as a helpful sign to check the stability of the private test set, which makes it easier to compare different model versions (more details are in Section 5.2).

4.5 Cross Validation Mechanism

The 5-Fold Cross-Validation process in this study follows a specific order of steps within each fold. First, tokenization is applied to the training and validation sets for that specific fold. Next, the optimizer, and scheduler are restarted from the pre-trained checkpoint to ensure that every fold is completely independent. Then, the model is trained using the validation set's macro-F1 score as the rule for early stopping. Finally, it generates the cross-validation (OOF) chances for the validation set and simultaneously makes predictions for the test set. The predicted chances for the test set from all 5 folds are then combined using a simple average.

After all folds are completely finished, a search for the best macro-F1 cut-off point (threshold) is conducted on the full set of OOF chances using a step size of 0.01. This specific cut-off point is then applied to the averaged chances of the test set to make the final classification. For the Ensemble version (Version 6.6), there is an extra step: the OOF chances and the test set chances produced by the three different models are first averaged together using equal weights. After that, a single cut-off point search is applied to these combined results.

4.6 Ablation Studies

This study conducted five ablation experiments on the Version 6 series to verify that the choices for each hyperparameter were reasonable.

4.6.1 Pseudo Loss Weight (Version 6, 6.1 and 6.2)

The best value for the weight is 0.3. Setting the weight to 0.5 is too high, causing the pseudo labels to dominate the loss calculation. This shifts the model's probabilities and pushes the best cut-off point to 0.71. On the other hand, setting the weight to 0.2 reaches the exact same result as 0.3, so lowering it further does not have any real effect (Table 6).

Table 6. Pseudo Loss Weight Ablation Results

Version	weight	OOF F1	Best Threshold
Version 6	0.50	0.8577	0.710
Version 6.1	**0.30**	**0.8694**	**0.610**
Version 6.2	0.20	0.8694	0.610

4.6.2 Confidence Threshold (Version 6.1 vs 6.3)

Table 7 shows the changes in the number of pseudo labels and the OOF F1 score after lowering the confidence cut-off point. Specifically, lowering the cut-off point doubled the number of pseudo samples (an increase of 108.8%). However, the samples added from the 0.90 to 0.92 confidence range brought in extra noise. Because of this, the OOF F1 score actually dropped. This result proves that the quality of the data is more important than quantity.

Table 7. Confidence Threshold Ablation Results

Version	ct	Number of Pseudo Samples	OOF F1
Version 6.1	**0.92**	**814**	**0.8694**
Version 6.3	0.90	1,700	0.8684

4.6.3 Cyclic Self-Training Stage3(Version 6.1 vs 6.4)

In Stage 3, the model used predictions from Stage 2 to generate new pseudo labels and train again. Although the cross-validation (OOF) F1 score increased by 0.0129 and seemed effective, the Public Score actually dropped by 0.69 percentage points. This is a classic case of circular confirmation bias. The model simply amplified its own biases and mistakes. Because the OOF evaluation was also produced by the same model, it could not detect these growing errors. The lesson learned here is that the self-training process should stop completely after Stage 2 (Table 8).

Table 8. Iterative Self-Training Ablation Results

Version	OOF F1	Public Score	OOF-Public Gap
Version 6.1 (Stage 2)	0.8694	**0.79173**	0.0777
Version 6.4 (Stage 3)	**0.8823**	0.78484	0.0975

4.6.4 Architecture Diversity (Version 6.1 vs 6.5)

Table 9 shows the differences between the two model designs in their cross-validation (OOF) and Public scores. The single DeBERTa model scored 0.0103 higher than RoBERTa on the OOF evaluation. However, its Public Score was actually 0.003 lower. This shows that simply using one stronger model design does not always mean it will handle new data better. Having a variety of different model designs is the real key to making the Ensemble work well.

Table 9. Architecture Diversity Ablation Results

Version	Model	OOF F1	Public Score
Version 6.1	RoBERTa-large	0.8694	0.79173
Version 6.5	DeBERTa-v3-large	0.8797	0.78870

4.6.5 ELECTRA Learning Rate Stability

When the ELECTRA model was first trained with a learning rate of 1e-5, Fold 3 crashed during Stage 1 with an F1 score of 0.030. As a result, the Stage 2 cross-validation (OOF) F1 score was only 0.7801. After the learning rate was adjusted to 5e-6, the OOF F1 score successfully increased to 0.8481 (more details can be found in Section 4.3.3). This result clearly proves that different early training methods require different learning rate settings.

4.7 Final Ensemble Configuration

The comparison of the three Ensemble configurations is shown in Table 10. As seen in the table, using an equal weight average performs better than the other setups in both the cross-validation (OOF) F1 score and the Positive Rate. Furthermore, adding the ELECTRA model (which only had a single-model OOF F1 score of 0.8481) actually improved the overall ensemble performance. This result clearly proves the hypothesis that having a low error correlation between models is much more important than the strength of each individual model.

Table 10. Final Ensemble Configuration Comparison

Configuration	OOF F1	OOF Acc	Threshold	Positive Rate
2-model (RoBERTa + DeBERTa)	0.8844	0.8845	0.560	0.507
3-model equal (1/3 each)	**0.8864**	**0.8865**	**0.610**	**0.487**
3-model weighted (0.4/0.4/0.2)	0.8860	0.8860	0.570	0.504

5. RESULT

5.1 Pre-Version Performance Overview

This research went through six rounds of version updates. The cross-validation (OOF) and Public Leaderboard scores for these versions are shown in Table 11.

Table 11. Per-version performance summary

Version	Strategy	OOF F1	OOF Acc	Public Score
Version 1	TF-IDF + Logistic Regression	0.6622	0.6665	—
Version 2	Word + Char TF-IDF	0.6697	0.6735	0.66308
Version 3	DistilBERT fine-tune	0.8196	0.8033	0.70137
Version 3.1	DistilBERT + Regularization	0.8239	0.8133	—
Version 4	RoBERTa-base + 5-Fold	0.8348	0.8305	0.74683
Version 5	RoBERTa-large + Pseudo-labeling	0.8715	0.8695	0.74710
Version 6	Two-Stage Self-Training	0.8546	0.8520	0.78099
Version 6.1	Two-Stage Self-Training + pseudo_loss_weight = 0.3	0.8694	0.8657	0.79173
Version 6.4	Two-Stage Self-Training + Stage 3 (Iterative Self-Training)	0.8823	0.8810	0.78484
Version 6.5	DeBERTa-v3-large	0.8797	0.8768	0.78870
Version 6.6	**3-Model Ensemble**	**0.8864**	**0.8865**	**0.79807**

The changes in the Public Score shown in Figure 6 clearly highlight three important turning points. First, the jump from Version 2 to Version 3 was a big leap (+0.038) because it introduced a semantic model. This proves that the understanding of meaning provided by a pre-trained Transformer is much higher than what simple word-counting models can ever reach. Second, the drop from Version 6.1 to Version 6.4 (-0.007) was caused by iterative self-training. Even though the cross-validation (OOF) F1 score went up by 0.013, the Public Score still went down. This reveals the risky, two-sided nature of self-training. Finally, Version 6.6, which combined three different models, achieved the highest score of 0.79807. This version also showed the best match between its OOF score and its true Public Score compared to all previous versions.

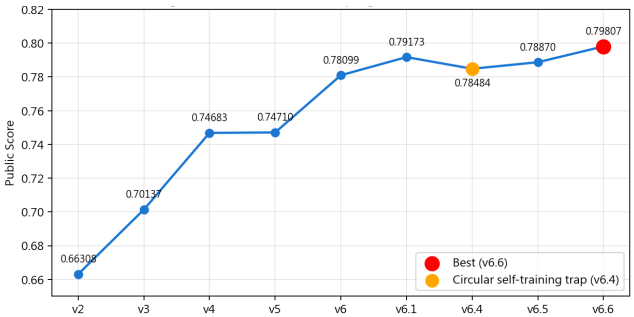

Figure 6. Evolution of Public Leaderboard Scores from Version 2 to Version 6.6

5.2 Final Model Evaluation Metrix

The final model, Version 6.6 (a three-model ensemble with equal weights), achieved the key markers shown in Table 12 during the 5-fold cross-validation.

The model reached a cross-validation (OOF) macro-F1 score of 0.8864 and an accuracy of 0.8865. The Public Leaderboard score was 0.79807, and the final Private Leaderboard score was **0.81099**. The Public and Private scores moved in the same direction, and the Private score was actually higher. This shows that the model's ability to handle new, unseen data on the full test set was better than the estimate provided by the 33% Public subset.

Therefore, the improvements seen in Version 6.6 came from a real increase in the model's capability, not just by chance. Additionally, the predicted positive rate of 0.487 is very close to the original training set's distribution (which is 0.500). This means that the cut-off point is healthy and well-adjusted, and the model's predicted probabilities did not drift noticeably.

Table 12. 5-Fold Cross-Validation Evaluation Metrics for Version 6.6

Evaluation Indicator	Score
OOF Macro-F1	0.8864
OOF Accuracy	0.8865
Best Threshold	0.610
Predicted Positive Rate	0.487
Public Leaderboard Score	0.79807
Private Leaderboard Score	**0.81099**

5.3 Generalization Analysis: OOF–Public Gap

To measure how well each version handles new data, this study compared them using the OOF-Public Gap defined in Section 4.4.4, as shown in Figure 7. The OOF F1 score and the Public Score generally moved together in the same direction. However, Version 6.4 clearly broke away from this trend, as its OOF score went up while its Public Score went down. This is a classic example of the iterative self-training trap. Version 6.6 achieved both the highest OOF score and the highest Public Score. Its gap of 0.0884 was

smaller than the gaps for Version 6.4 (0.0975) and Version 6.5 (0.0910), showing that its OOF evaluation was much closer to its true performance. The three-model ensemble combined three different early training methods: MLM, Disentangled MLM, and RTD. Because these methods tend to make different kinds of mistakes, they successfully reduced the chance of errors pulling the model in the same wrong direction. This makes Version 6.6 the best version for closely matching the cross-validation (OOF) evaluation with the true Public score.

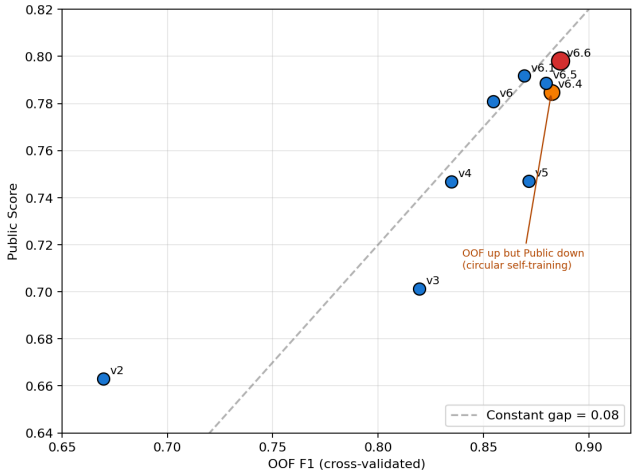

Figure 7. OOF F1 and Public Score scatter plot; the dashed line is the reference line with "equidistant gap = 0.08".

5.4 Final Submission

The final submission file is Version 6.6, which contains 11,000 predictions for the test set. The results from the cross-validation (OOF) F1 score of 0.8864, the Public Leaderboard score of 0.79807, and the Private Leaderboard score of 0.81099 all strongly align in the same direction. Furthermore, the Private score is actually higher than the Public score. This clearly shows that the model's ability to handle new data on the complete test set is very stable and that no overfitting has occurred. The strategy of using a three-model ensemble with equal weights, combined with adjusting the cut-off point based on the OOF evaluation, successfully turned the variety of different models into a real improvement in the final ranking.

6. CONCLUSION

This research started from a TF-IDF bag-of-words baseline, gradually introduced pre-trained Transformers and a two-stage self-training framework, and finally achieved a cross-validation (OOF) F1 score of 0.8864 and a Private Leaderboard score of 0.81099 using an equal-weight ensemble of three models: RoBERTa, DeBERTa, and ELECTRA. This represents an improvement of over 13.5 percentage points compared to the original bag-of-words baseline.

The experiments led to three main conclusions. First, the understanding of meaning provided by a pre-trained Transformer is a level that bag-of-words models can never reach. Simply introducing DistilBERT brought the single largest jump in performance (+0.1499). Second, pseudo-labels must be paired with strict confidence filtering and stopping rules. Although the iterative self-training in Version 6.4 increased the OOF F1 score, the Public Score actually went down. This clearly shows that training too much on the model's own predictions will trap it in confirmation bias. Third, having a variety of different model designs is much more effective than just using a single stronger model. The three different early training methods caused the models to make mistakes in completely non-overlapping ways. Because of this, the combined ensemble consistently performed better than any single model alone.

Future work could try adding even more different model designs to expand the variety of the ensemble. Another option is to add cross-model consistency checks when filtering pseudo-labels to reduce noise even further.

7. REFERENCES

[1] T. Wolf *et al.*, "Transformers: State-of-the-Art Natural Language Processing," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, Online: Association for Computational Linguistics, Oct. 2020, pp. 38–45. doi: 10.18653/v1/2020.emnlp-demos.6.

[2] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Inf. Process. Manag.*, vol. 24, no. 5, pp. 513–523, Jan. 1988, doi: 10.1016/0306-4573(88)90021-0.

[3] A. Vaswani *et al.*, "Attention Is All You Need," in *Advances in Neural Information Processing Systems 30*, Long Beach, CA, USA, 2017, pp. 6000–6010.

[4] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, Minneapolis, Minnesota, Jun. 2019, pp. 4171–4186. doi: 10.18653/v1/N19-1423.

[5] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter," Mar. 01, 2020, *arXiv*: arXiv:1910.01108. doi: 10.48550/arXiv.1910.01108.

[6] Y. Liu *et al.*, "RoBERTa: A Robustly Optimized BERT Pretraining Approach," Jul. 26, 2019, *arXiv*: arXiv:1907.11692. doi: 10.48550/arXiv.1907.11692.

[7] S. Narang *et al.*, "MIXED PRECISION TRAINING," presented at the ICLR 2018 Conference, 2018.

[8] T. G. Dietterich, "Ensemble Methods in Machine Learning," in *Multiple Classifier Systems*, vol. 1857, in Lecture Notes in Computer Science, vol. 1857. , Berlin, Heidelberg: Springer Berlin Heidelberg, 2000, pp. 1–15. doi: 10.1007/3-540-45014-9_1.

[9] P. He, J. Gao, and W. Chen, "DEBERTAV3: IMPROVING DEBERTA USING ELECTRA-STYLE PRE-TRAINING WITH GRADIENT- DISENTANGLED EMBEDDING SHARING," presented at the ICLR 2023 poster, Feb. 2023.

[10] K. Clark, M.-T. Luong, and Q. V. Le, "ELECTRA: PRE-TRAINING TEXT ENCODERS AS DISCRIMINATORS RATHER THAN GENERATORS," in *ICLR 2020*, Sep. 2020.